C-DAC Mumbai Day 05 DBT

Part 1: Subqueries

```
mysql>  create table employee ( emp_id int primary key,
    ->      emp_name varchar (30),
    ->      salary decimal (10,2),
    ->      dpet_id int,
    ->      manager_id int,
    ->      constraint fk_department_dept_id
    ->      foreign key (dpet_id) references department(dpet_id));
Query OK, 0 rows affected (0.03 sec)

mysql> desc employee;
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| emp_id     | int           | NO   | PRI | NULL    |       |
| emp_name   | varchar(30)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| dpet_id    | int           | YES  | MUL | NULL    |       |
| manager_id | int           | YES  |     | NULL    |       |
+------------+---------------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql> insert into department values (10, 'IT'),
    -> (20, 'Sales'), (30,'HR'), (40,'research');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> select * from department;
+---------+-----------+
| dpet_id | dept_name |
+---------+-----------+
```

1. What is a subquery?

A subquery is a query written inside another SQL query. The inner query returns a result that the outer query uses.

```
SELECT emp_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary)
```

```
    FROM employee
);
```

- The inner query calculates the average salary.
- The outer query returns employees whose salaries are above that average.

2. Types of subqueries

A. Scalar subquery

Returns exactly one value: one row and one column. It is useful for comparisons with AVG, MAX or MIN.

```
SELECT emp_name, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);
```

```
mysql> select emp_name, salary from employee where salary > ( select avg(salary) from employee);
+----------+----------+
| emp_name | salary   |
+----------+----------+
| Omkar    | 70000.00 |
| Abhishek | 65000.00 |
| Aditya   | 80000.00 |
+----------+----------+
3 rows in set (0.01 sec)

mysql> select emp_name, salary from employee where
    ->  (salary, dept_id) = ( select salary, dept_id from employee where emp_name='Aniket');
ERROR 1054 (42S22): Unknown column 'dept_id' in 'where clause'
mysql> select emp_name, salary from employee where
    ->  (salary, dpet_id) = ( select salary, dpet_id from employee where emp_name='Aniket');
+----------+----------+
| emp_name | salary   |
+----------+----------+
| Aniket   | 60000.00 |
+----------+----------+
```

B. Single-row subquery

Returns one row, potentially with multiple columns. The outer comparison must match the number of returned columns.

This finds employees whose salary and department both match Amit's. Amit himself is also included.

Another example, comparing the maximum salary and department ID 10:

```
SELECT emp_name, salary
FROM employee
WHERE (salary, dept_id) = (
    SELECT MAX(salary), 10
    FROM employee
);
```

C. Multi-row subquery

Returns multiple rows, commonly used with IN, ANY or ALL.

```
SELECT emp_name, dept_id
FROM employee
WHERE dept_id IN (
    SELECT dept_id FROM department
```

```
    WHERE dept_name IN ('IT', 'Sales')
);
```

```
mysql> select emp_name, salary, dpet_id from employee
    -> where dpet_id in  (select dpet_id from department where  dept_name in ('IT', 'Sales'));
+----------+----------+---------+
| emp_name | salary   | dpet_id |
+----------+----------+---------+
| Aniket   | 60000.00 |      10 |
| Abhishek | 65000.00 |      10 |
| Aniket   | 60000.00 |      10 |
| Pranav   | 50000.00 |      20 |
| Aditya   | 80000.00 |      20 |
+----------+----------+---------+
5 rows in set (0.00 sec)
```

D. Multi-column subquery

Returns two or more columns. A multi-column subquery may return one row or multiple rows.

```
SELECT emp_name, salary, dept_id
FROM employee
WHERE (salary, dept_id) IN (
    SELECT MAX(salary), dept_id
    FROM employee
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
);
```

This finds employees whose salary matches the maximum salary for their department. Subquery types can overlap: a multi-column subquery can also be single-row or multi-row.

3. Where can subqueries be used?

A. In WHERE

```
SELECT emp_name, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);
```

Filters rows based on the result of the inner query.

B. In FROM: derived table

A subquery in FROM acts as a result set for the outer query. MySQL requires an alias for it.

```
SELECT dept_id, avg_salary
FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employee
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
) AS dept_summary
WHERE avg_salary > 60000;
```

C. In SELECT

```
SELECT emp_name, salary,
       (SELECT AVG(salary) FROM employee) AS company_avg
FROM employee;
```

Each employee row displays the company average salary alongside that employee's salary.

4. IN, ANY and ALL

A. IN

IN checks whether a value matches any value returned by the subquery.

```
mysql> select emp_name, salary, dpet_id
    -> from employee
    -> where (salary, dpet_id) In   (select Min(salary),dpet_id
    -> from employee group by dpet_id);
+----------+----------+---------+
| emp_name | salary   | dpet_id |
+----------+----------+---------+
| Aniket   | 60000.00 |      10 |
| Pranav   | 50000.00 |      20 |
| Shivraj  | 50000.00 |      40 |
| Malkeet  | 45000.00 |      30 |
| Aniket   | 60000.00 |      10 |
+----------+----------+---------+
5 rows in set (0.00 sec)
```

B. ANY

The comparison is true if it is true for at least one returned value.

```
mysql> select emp_name, salary
    -> from employee
    -> where salary > Any ( select salary from employee where dpet_id=20);
+----------+----------+
| emp_name | salary   |
+----------+----------+
| Aniket   | 60000.00 |
| Omkar    | 70000.00 |
| Abhishek | 65000.00 |
| Aditya   | 80000.00 |
| Aniket   | 60000.00 |
+----------+----------+
5 rows in set (0.00 sec)
```

.

C. ALL

The comparison must be true for every returned value.

```
mysql> select emp_name, salary
    -> from employee
    -> where salary > All (select salary from employee where dpet_id=10);
+----------+----------+
| emp_name | salary   |
+----------+----------+
| Omkar    | 70000.00 |
| Aditya   | 80000.00 |
+----------+----------+
2 rows in set (0.00 sec)
```

This finds employees earning more than every HR employee. Here, salary must be greater than 55000.

Operator	Meaning
IN	Value matches one of the returned values.
> ANY	Value is greater than at least one returned value.
> ALL	Value is greater than every returned value.

5. NOT IN and the NULL trap

If a NOT IN subquery returns NULL, a comparison that otherwise seems true can evaluate to UNKNOWN. WHERE filters out UNKNOWN results.

```
SELECT 20 NOT IN (10, NULL);
-- Result: NULL (unknown), not TRUE
```

NOT EXISTS is often safer when checking that no matching row exists.

```
SELECT e.emp_name, e.dept_id
FROM employee e
WHERE NOT EXISTS (
    SELECT 1
    FROM department d
    WHERE d.dept_id = e.dept_id
);
```

This returns employees for whom no matching department row exists, including an employee whose dept_id is NULL.

6. Correlated subqueries

A correlated subquery refers to a column from the outer query. This example finds employees earning more than the average salary in their own department.

```
mysql> select e.emp_name, e.salary, e.dpet_id
    -> from employee e
    -> where e.salary >  ( select avg(salary) from employee e2
    -> where e.dpet_id= e2.dpet_id);
+----------+----------+---------+
| emp_name | salary   | dpet_id |
+----------+----------+---------+
| Omkar    | 70000.00 |      30 |
| Abhishek | 65000.00 |      10 |
| Aditya   | 80000.00 |      20 |
+----------+----------+---------+
3 rows in set (0.00 sec)
```

The condition e2.dept_id = e.dept_id connects the inner query to the outer row. You can explain it as if the inner query were checked for each outer row, but MySQL may optimize it and does not necessarily execute it separately for every row.

8. Subquery vs JOIN

Both can solve the same problem. Example: find employees who work in IT.

Using a subquery

```
SELECT emp_name
FROM employee
WHERE dept_id IN (
    SELECT dept_id FROM department
    WHERE dept_name = 'IT'
);
```

Using a JOIN

```
SELECT e.emp_name
FROM employee e
JOIN department d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'IT';
```

Use a subquery when…	Use a JOIN when…
Comparing with an aggregate such as an average.	You need columns from multiple tables.
Checking whether matching rows exist.	Combining related table data.
A named intermediate result makes the logic clearer.	Displaying details from both tables.

A subquery is not automatically slower or faster than a JOIN. Compare query plans when performance matters.

Part 2: TCL and implicit commits

9. What is TCL?

TCL stands for Transaction Control Language. It controls transactions in a database.

Command	Purpose
START TRANSACTION	Starts a transaction.
COMMIT	Saves transaction changes permanently.

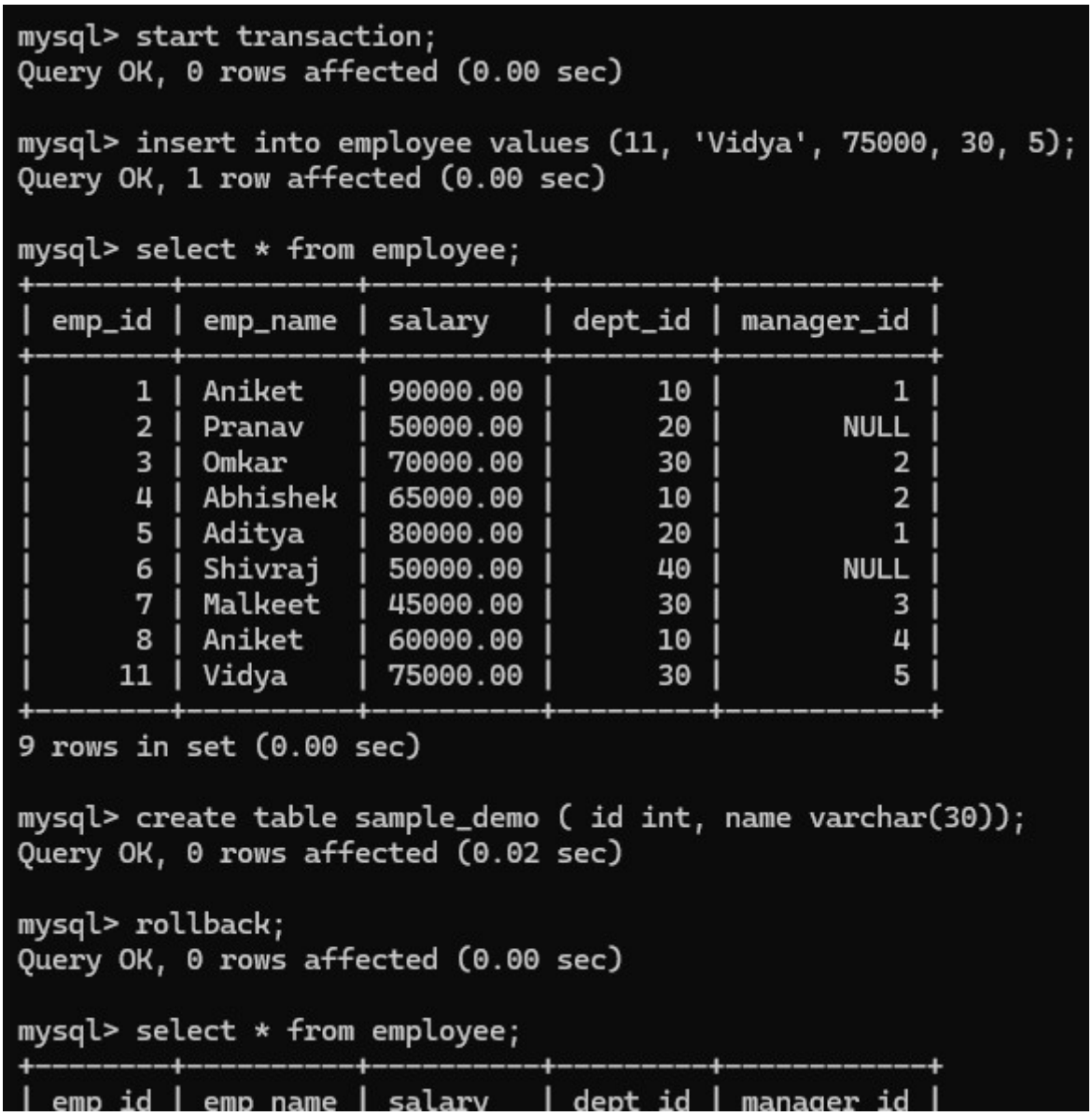
ROLLBACK	Cancels uncommitted transactional changes.
SAVEPOINT	Creates a point to which you can roll back.
ROLLBACK TO SAVEPOINT	Rolls back to a savepoint without cancelling the whole transaction.

10. Implicit commits caused by DDL

DDL means Data Definition Language and includes CREATE, ALTER and DROP. In MySQL, many DDL statements cause an implicit commit.

Do not expect ROLLBACK to undo the salary update. CREATE TABLE causes an implicit commit, so the preceding update is committed. The table creation also occurs.

- ROLLBACK cannot undo changes that have already been committed.
- Avoid mixing DDL into a transaction when you expect to roll back all changes.



```
mysql> start transaction;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into employee values (11, 'Vidya', 75000, 30, 5);
Query OK, 1 row affected (0.00 sec)

mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
+--------+----------+----------+---------+------------+
|      1 | Aniket   | 90000.00 |      10 |          1 |
|      2 | Pranav   | 50000.00 |      20 |       NULL |
|      3 | Omkar    | 70000.00 |      30 |          2 |
|      4 | Abhishek | 65000.00 |      10 |          2 |
|      5 | Aditya   | 80000.00 |      20 |          1 |
|      6 | Shivraj  | 50000.00 |      40 |       NULL |
|      7 | Malkeet  | 45000.00 |      30 |          3 |
|      8 | Aniket   | 60000.00 |      10 |          4 |
|     11 | Vidya    | 75000.00 |      30 |          5 |
+--------+----------+----------+---------+------------+
9 rows in set (0.00 sec)

mysql> create table sample_demo ( id int, name varchar(30));
Query OK, 0 rows affected (0.02 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
```

Part 3: DCL — Data Control Language

11. What is DCL?

DCL controls what database users are allowed to do. GRANT gives privileges; REVOKE removes privileges.

12. Create a user

```
CREATE USER 'report_user'@'localhost'
IDENTIFIED BY 'StrongPassword123!';
```

This account is for connections from the local machine. Use a strong password and configure the host appropriately.

13. GRANT, REVOKE and SHOW GRANTS

A. Give read-only access to a database

```
GRANT SELECT ON company_db.*
TO 'report_user'@'localhost';
```

B. Give INSERT and UPDATE on one table

```
GRANT INSERT, UPDATE ON company_db.employee
TO 'report_user'@'localhost';
```

C. Revoke UPDATE on one table

```
REVOKE UPDATE ON company_db.employee
FROM 'report_user'@'localhost';
```

This removes UPDATE granted at that scope; other applicable privileges remain.

D. Show a user's privileges

```
SHOW GRANTS FOR 'report_user'@'localhost';
```

14. WITH GRANT OPTION

WITH GRANT OPTION allows a user to grant onward the privileges granted to them, subject to MySQL privilege rules.

```
GRANT SELECT ON company_db.*
TO 'report_user'@'localhost'
WITH GRANT OPTION;
```

To remove the ability to grant privileges onward:

```
REVOKE GRANT OPTION ON company_db.*
FROM 'report_user'@'localhost';
```

This is different from revoking SELECT itself.

15. Privilege levels

Scope	Example	Meaning
Global	ON *.*	Applies across databases.
Database	ON company_db.*	Applies to tables in one database.
Table	ON company_db.employee	Applies to one table.
Column	ON company_db.employee (salary)	Applies to a specified column for supported privileges.

A column-level grant permits access to specified columns through that grant. Other applicable grants can affect effective permissions. Demonstrate DCL using a separate test account, not root.

```
mysql> create user 'secondUser'@'localhost' identified by 'Welcome@123';
Query OK, 0 rows affected (0.02 sec)

mysql> select user, host from mysql.user;
+------------------+-----------+
| user             | host      |
+------------------+-----------+
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
| secondUser       | localhost |
+------------------+-----------+
5 rows in set (0.00 sec)

mysql> grant all privileges on *.* to 'secondUser'@'localhost' with grant option;
Query OK, 0 rows affected (0.05 sec)

mysql> show grants for
    -> 'secondUser'@localhost;
```

```
C:\Users\aniket>mysql -u seconduser -p
Enter password: ***********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 24
Server version: 8.0.31 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| performance_schema |
+--------------------+
2 rows in set (0.01 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| oct13              |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)
```

```
Database changed
mysql> show tables;
+----------------+
| Tables_in_oct13 |
+----------------+
| employee       |
+----------------+
1 row in set (0.00 sec)

mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
+--------+----------+----------+---------+------------+
|      1 | Aniket   | 90000.00 |      10 |          1 |
|      2 | Pranav   | 50000.00 |      20 |       NULL |
|      3 | Omkar    | 70000.00 |      30 |          2 |
|      4 | Abhishek | 65000.00 |      10 |          2 |
|      5 | Aditya   | 80000.00 |      20 |          1 |
|      6 | Shivraj  | 50000.00 |      40 |       NULL |
|      7 | Malkeet  | 45000.00 |      30 |          3 |
|      8 | Aniket   | 60000.00 |      10 |          4 |
|     11 | Vidya    | 75000.00 |      30 |          5 |
+--------+----------+----------+---------+------------+
9 rows in set (0.00 sec)

mysql> insert into employee values (12, 'Ashish' , 66000, 10, null);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> grant insert, select on oct13.employee
    -> to 'seconduser'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
+--------+----------+----------+---------+------------+
|      1 | Aniket   | 90000.00 |      10 |          1 |
|      2 | Pranav   | 50000.00 |      20 |       NULL |
|      3 | Omkar    | 70000.00 |      30 |          2 |
|      4 | Abhishek | 65000.00 |      10 |          2 |
|      5 | Aditya   | 80000.00 |      20 |          1 |
|      6 | Shivraj  | 50000.00 |      40 |       NULL |
|      7 | Malkeet  | 45000.00 |      30 |          3 |
|      8 | Aniket   | 60000.00 |      10 |          4 |
|     11 | Vidya    | 75000.00 |      30 |          5 |
|     12 | Ashish   | 66000.00 |      10 |       NULL |
+--------+----------+----------+---------+------------+
10 rows in set (0.00 sec)

mysql> grant update on oct13.employee
    -> to 'seconduser'@localhost;
Query OK, 0 rows affected (0.00 sec)

mysql> show grants for 'seconduser'@localhost;
+-----------------------------------------------------------------------------+
| Grants for seconduser@localhost                                             |
+-----------------------------------------------------------------------------+
```

```
mysql> update employee set salary =86000 where emp_id=3;
ERROR 1142 (42000): UPDATE command denied to user 'seconduser'@'localhost' for table 'employee'
mysql> update employee set salary =86000 where emp_id=3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> create user 'thirduser'@'localhost' identified by 'Welcome@123';
Query OK, 0 rows affected (0.01 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| aug25              |
| aug26              |
| index_demo         |
| information_schema |
| mysql              |
| oct13              |
```

Part 4: Views

MySQL Views

A view in MySQL is a virtual table defined by a SELECT statement. MySQL stores the view definition, not normally a separate copy of the result rows. When the view is queried, MySQL uses the definition to retrieve the data from the underlying table or tables.

1. Why do we use views?

Views make repeated queries easier to reuse, present data in a simpler form, combine data from multiple tables, and expose only selected columns or rows to users. A view alone does not guarantee better performance or security; permissions must be configured correctly.

3. Create and query a view

The view returns the selected columns. It does not normally maintain an independent copy of those rows.

4. Filtered view

```
create view sales_employees as
  select emp_id, emp_name, salary
  from employee where dept_id =20;

SELECT * FROM sales_employees;
```

```
mysql> select * from sales_employees;
+--------+----------+----------+
| emp_id | emp_name | salary   |
+--------+----------+----------+
|      2 | Pranav   | 50000.00 |
|      5 | Aditya   | 80000.00 |
+--------+----------+----------+
2 rows in set (0.00 sec)

mysql> insert into sales_employees ( emp_id,emp_name,  salary )
    ->   values ( 15, 'Vipul', 74000);
ERROR 1062 (23000): Duplicate entry '15' for key 'employee.PRIMARY'
mysql> insert into sales_employees ( emp_id,emp_name,  salary )
    ->   values ( 16, 'Vipul', 74000);
Query OK, 1 row affected (0.01 sec)

mysql> select * from employees;
ERROR 1146 (42S02): Table 'oct13.employees' doesn't exist
mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
+--------+----------+----------+---------+------------+
|      1 | Aniket   | 90000.00 |      10 |          1 |
```

The condition in the view definition is still applied, and a query can add further conditions.

5. View based on multiple tables

```
create view sales_employee_with_dept_name as

  select e.emp_id, e.emp_name, e.salary, d.dept_name

  from employee e , department d

  where e.dept_id=d.dept_id and e.dept_id=20;

SELECT * FROM employee_department_view;
```

This view saves a commonly used join between employee and department.

```
mysql> select * from sales_employee_with_dept_name;
+--------+----------+----------+-----------+
| emp_id | emp_name | salary   | dept_name |
+--------+----------+----------+-----------+
|      2 | Pranav   | 50000.00 | Sales     |
|      5 | Aditya   | 80000.00 | Sales     |
+--------+----------+----------+-----------+
2 rows in set (0.00 sec)

mysql> update sales_employee_with_dept_name set salary =95000 where emp_id=2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
+--------+----------+----------+---------+------------+
|      1 | Aniket   | 90000.00 |      10 |          1 |
|      2 | Pranav   | 95000.00 |      20 |       NULL |
|      3 | Omkar    | 86000.00 |      30 |          2 |
|      4 | Abhishek | 65000.00 |      10 |          2 |
```

6. INSERT, UPDATE and DELETE through a view

If a view is updatable and the operation satisfies the view definition, table constraints and permissions, modifying data through the view affects the underlying table. It does not modify a separate copy of the data.

```
create view sales_employees as

  select emp_id, emp_name, salary

  from employee where dept_id =20;



  create view sales_employee_with_dept_name as

  select e.emp_id, e.emp_name, e.salary, d.dept_name

  from employee e , department d

  where e.dept_id=d.dept_id and e.dept_id=20;


  insert into sales_employees ( emp_id,emp_name,  salary )

  values ( 16, 'Vipul', 74000);.
```

7. Which views are updatable?

A simple view based on one table, without grouping or aggregate calculations, is often updatable. Updatability depends on the exact definition. Views using GROUP BY, aggregate functions (such as AVG or COUNT), DISTINCT, UNION, or certain joins and expressions are generally not updatable or have restrictions.

The average is a calculated result for a group of rows, so MySQL cannot map an update of that average to one specific employee salary.

```
mysql> select * from sales_employee_with_dept_name;
+--------+----------+----------+-----------+
| emp_id | emp_name | salary   | dept_name |
+--------+----------+----------+-----------+
|      2 | Pranav   | 50000.00 | Sales     |
|      5 | Aditya   | 80000.00 | Sales     |
+--------+----------+----------+-----------+
2 rows in set (0.00 sec)

mysql> update sales_employee_with_dept_name set salary =95000 where emp_id=2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from employee;
+--------+----------+----------+---------+------------+
| emp_id | emp_name | salary   | dept_id | manager_id |
+--------+----------+----------+---------+------------+
|      1 | Aniket   | 90000.00 |      10 |          1 |
|      2 | Pranav   | 95000.00 |      20 |       NULL |
|      3 | Omkar    | 86000.00 |      30 |          2 |
|      4 | Abhishek | 65000.00 |      10 |          2 |
```

```
mysql> create view Average_salary_per_dept as
    ->   select dept_id, avg(salary) from employee group by dept_id;
Query OK, 0 rows affected (0.01 sec)

mysql> select * from Average_salary_per_dept;
+---------+--------------+
| dept_id | avg(salary)  |
+---------+--------------+
|    NULL | 74000.000000 |
|      10 | 70250.000000 |
|      20 | 87500.000000 |
|      30 | 68666.666667 |
|      40 | 50000.000000 |
+---------+--------------+
5 rows in set (0.00 sec)

mysql> update Average_salary_per_dept
    ->   set avg(salary) = 71000 where dept_id=10;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use
 near '(salary) = 71000 where dept_id=10' at line 2
```

8. WITH CHECK OPTION

```
CREATE OR REPLACE VIEW it_employees AS
SELECT emp_id, emp_name, salary, dept_id
FROM employee
WHERE dept_id = 10
WITH CHECK OPTION;
```

```
-- MySQL rejects this through the view because dept_id would no longer be 10
UPDATE it_employees
SET dept_id = 20
WHERE emp_id = 1;
```

WITH CHECK OPTION prevents INSERT or UPDATE through an updatable view when the resulting row would not satisfy the view's WHERE condition.

9. Display, replace and drop a view

```
-- Show the view definition
SHOW CREATE VIEW sales_employees;

-- Replace its definition
CREATE OR REPLACE VIEW sales_employees AS
SELECT emp_id, emp_name, salary
FROM employee
WHERE dept_id = 10;

-- Remove only the view definition
DROP VIEW sales_employees;
```

DROP VIEW removes the view, not the underlying employee table or its records.

10. Views and permissions

A view can omit sensitive columns, such as salary, from the result exposed to a user. To use this as a security boundary, grant suitable permissions on the view and avoid giving that user broader access to the underlying table. MySQL view definitions can use SQL SECURITY DEFINER or SQL SECURITY INVOKER, which affect whose privileges are checked.

11. Advantages and limitations

Advantages: reuse SQL logic; simplify queries; present selected rows and columns; combine related tables; help control data access when privileges are configured properly.

Limitations: not all views are updatable; views do not automatically improve performance; complex definitions can be harder to maintain; underlying table changes can affect a view; permissions must be configured carefully.

12. Quick comparison

View	Underlying table
Stores a saved SELECT definition	Stores the actual table data
Normally does not store a separate copy of result rows	Stores rows and columns
DROP VIEW removes the view definition	DROP TABLE removes the table and its data
May allow updates if updatable	Supports data changes subject to constraints and permissions

21. Advantages of views

Advantage	Example
Security	Expose employee names without exposing salary columns.
Abstraction	Hide complex joins behind a simple view name.
Reuse	Use the same view in multiple reports.
Simplicity	Query a summary instead of rewriting aggregate logic.
Consistency	Keep commonly used query logic in one definition.

13. Student practice questions

1. What is a view in MySQL?
2. Does a normal view store a separate copy of the result rows?
3. Can a view be created using more than one table?
4. What happens to the underlying table when data is updated through an updatable view?
5. Why are views using GROUP BY or AVG() generally not updatable?
6. What is the purpose of WITH CHECK OPTION?
7. What is the difference between DROP VIEW and DROP TABLE?
8. Does creating a view automatically improve query performance?
9. Find employees earning more than the company's average salary.
10. Find employees earning more than their department's average salary.
11. Find employees who work in IT or Sales using a subquery.
12. Find departments that have no employees using NOT EXISTS.
13. Explain why NOT IN may behave unexpectedly when a subquery returns NULL.
14. Create a user and grant it read-only access to company_db.
15. Create a view showing employee names and department names.
16. Create a view showing employees earning at least ₹60,000.
17. Explain why a view containing GROUP BY is generally non-updatable.
18. Explain what happens when a DDL statement causes an implicit commit during a transaction.